

Assignment 4 — DATA301

P. Javier Lopez Dondarza

Contents

Exercise 1	1
1.1 Sample quantiles	1
1.2 Estimators and bootstrap bias correction	2
1.3 Monte Carlo experiment	3
Exercise 2	5
References	12

```
library(fpp2)
library(ggplot2)
library(gridExtra)
set.seed(301)
```

Exercise 1

1.1 Sample quantiles

Wicklin (2017) compares the nine ways of computing a sample quantile that Hyndman & Fan (1996) list. They all interpolate between two neighbouring order statistics, and the only thing that changes is where the interpolation point sits. Types 1 to 3 are step functions that only return values from the sample. Types 4 to 9 are continuous and linear between order statistics. Type 6 is the default in Minitab and SPSS, type 2 in SAS, and type 7 in R. For large n they all give about the same answer, but for small n (and for p close to 0 or 1) they can differ quite a bit.

I would use type 8. Hyndman & Fan recommend it because it is approximately median-unbiased whatever the distribution is, and that matters here: the exponential is skewed and I am going to use $n = 5$. It is also continuous, so the IQR changes smoothly with the data. As a check I also run type 7, which is what R uses by default (`quantile(x, type = 7)`). NumPy's default (`method="linear"`) is type 7 as well.

```
quantile(c(2, 4, 4, 5, 9), c(.25, .75), type = 7)
```

```
## 25% 75%
##   4   5
```

```
quantile(c(2, 4, 4, 5, 9), c(.25, .75), type = 8)
```

```
##          25%          75%
## 3.333333 6.333333
```

1.2 Estimators and bootstrap bias correction

For an exponential variable the quantile function is $q_p = -\ln(1-p)/\lambda$, so

$$q_{3/4} - q_{1/4} = \frac{\ln 4 - \ln(4/3)}{\lambda} = \frac{\ln 3}{\lambda},$$

which is where $\hat{\lambda}_{IQR} = \ln 3 / \text{IQR}$ comes from.

If $\hat{\lambda}_{IQR}$ is biased, then $b = \mathbb{E}[\hat{\lambda}_{IQR}] - \lambda$. The bootstrap idea is to replace the unknown distribution by the empirical one. I take B resamples X^{*b} of size n with replacement, compute $\hat{\lambda}_{IQR}^{*b}$ for each, and estimate the bias by

$$\hat{b}_{boot} = \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR}^{*b} - \hat{\lambda}_{IQR}(X).$$

Subtracting it gives the corrected estimator

$$\tilde{\lambda} = \hat{\lambda}_{IQR} - \hat{b}_{boot} = 2\hat{\lambda}_{IQR}(X) - \frac{1}{B} \sum_{b=1}^B \hat{\lambda}_{IQR}^{*b}, \quad B = 200.$$

One practical problem: with $n = 5$ a resample can have a zero IQR (for example when the same value is drawn several times), which makes $\hat{\lambda}^*$ infinite. I drop the non-finite replicates before averaging.

```
QTYPE <- 8

lam_ml <- function(x) 1 / mean(x)

lam_iqr <- function(x, type = QTYPE) {
  q <- quantile(x, c(.25, .75), type = type, names = FALSE)
  log(3) / (q[2] - q[1])
}

lam_bc <- function(x, B = 200, type = QTYPE) {
  n <- length(x)
  est <- lam_iqr(x, type)
  boot <- replicate(B, lam_iqr(sample(x, n, replace = TRUE), type))
  boot <- boot[is.finite(boot)]
  2 * est - mean(boot)
}

x <- rexp(30, rate = 1)
c(ML = lam_ml(x), IQR = lam_iqr(x), BC = lam_bc(x))
```

```
##          ML          IQR          BC
## 0.8284973 1.0175734 0.9971686
```

1.3 Monte Carlo experiment

The metric is the mean square error, $\text{MSE}(\hat{\theta}) = \mathbb{E}[(\hat{\theta} - \lambda)^2]$, estimated by $\frac{1}{R} \sum_{r=1}^R (\hat{\theta}_r - \lambda)^2$.

How I chose R . The Monte Carlo standard error of that estimate is $\text{sd}(e^2)/\sqrt{R}$ with $e = \hat{\theta} - \lambda$ (Robert & Casella, 2009). I fixed a target: the standard error should be about 2% of the MSE itself. Solving for R gives $R = (\text{sd}(e^2)/(0.02 \text{MSE}))^2$, and I estimate $\text{sd}(e^2)$ and MSE with a pilot run of 500 replications. Two observations shape the design:

- All three estimators are scale equivariant, $\hat{\lambda}(cX) = \hat{\lambda}(X)/c$. That means MSE/λ^2 is the same for every λ , so the R I need depends on n only. The three values of λ then work as a check: the results should agree up to simulation noise.
- Small n needs more replications. Both $1/\bar{X}$ and $\hat{\lambda}_{IQR}$ have heavy right tails, so the squared errors are very spread out.

For each n I take the largest R required by the three estimators, so they can all use the same samples.

```
one_cell <- function(lambda, n, R, B = 200) {
  res <- matrix(NA_real_, R, 3, dimnames = list(NULL, c("ML", "IQR", "BC")))
  for (r in 1:R) {
    x <- rexp(n, lambda)
    res[r, ] <- c(lam_ml(x), lam_iqr(x), lam_bc(x, B))
  }
  res
}

pilot_R <- function(n, R0 = 500, prec = 0.02, cap = 20000) {
  e <- (one_cell(1, n, R0) - 1)^2
  need <- (apply(e, 2, sd) / (prec * colMeans(e)))^2
  min(cap, ceiling(max(need) / 500) * 500)
}

Rn <- sapply(c(5, 30, 100), pilot_R)
names(Rn) <- c(5, 30, 100)
Rn
```

```
##      5      30     100
## 20000 16500  9000
```

```
grid <- expand.grid(lambda = c(.1, 1, 10), n = c(5, 30, 100))

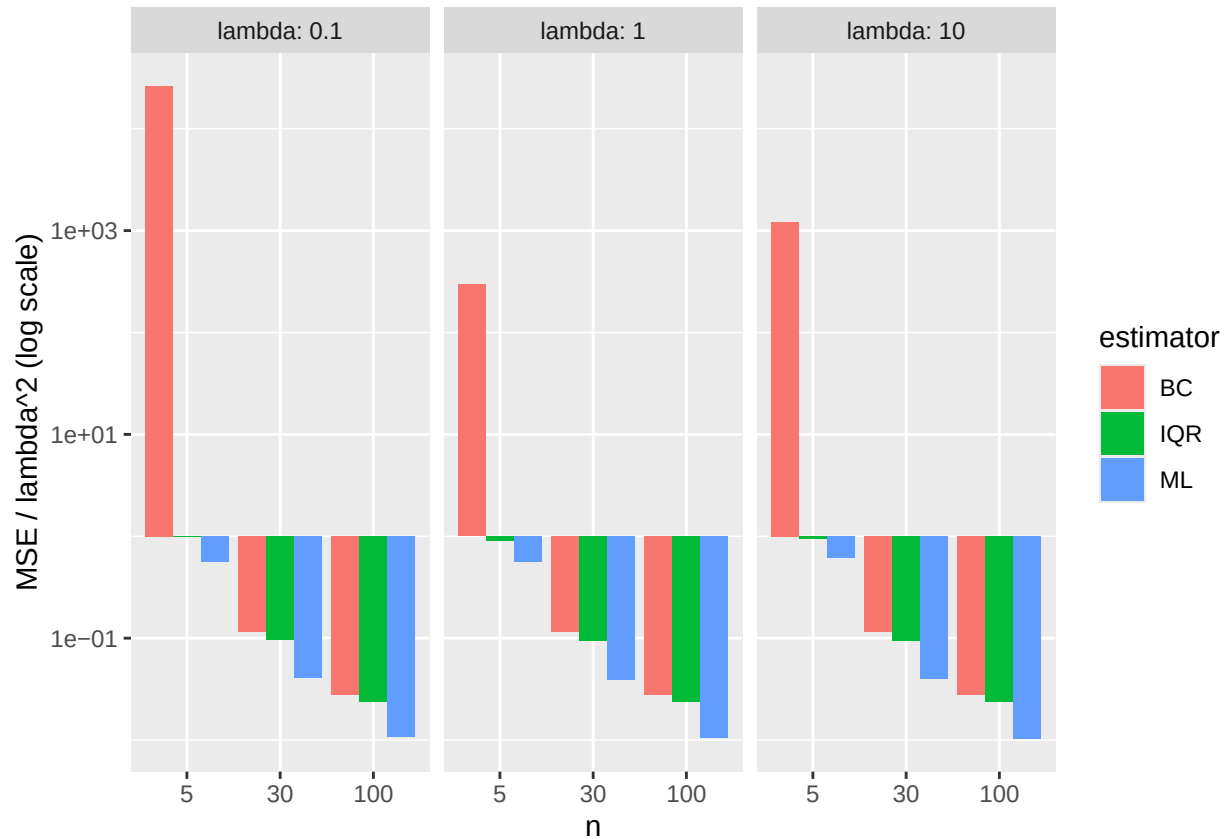
results <- do.call(rbind, lapply(seq_len(nrow(grid)), function(i) {
  lam <- grid$lambda[i]; n <- grid$n[i]; R <- Rn[as.character(n)]
  est <- one_cell(lam, n, R)
  e2 <- (est - lam)^2
  data.frame(lambda = lam, n = n, R = R,
    estimator = colnames(est),
    MSE = colMeans(e2),
    MC_se = apply(e2, 2, sd) / sqrt(R),
    relMSE = colMeans(e2) / lam^2)
}))

wide <- reshape(results[, c("lambda", "n", "R", "estimator", "MSE")],
```

```
idvar = c("lambda", "n", "R"), timevar = "estimator",
direction = "wide")
knitr::kable(wide, digits = 4)
```

	lambda	n	R	MSE.ML	MSE.IQR	MSE.BC
1	0.1	5	20000	0.0056	0.0100	263.4973
4	1.0	5	20000	0.5628	0.8957	295.2255
7	10.0	5	20000	62.0496	93.7607	121562.2300
10	0.1	30	16500	0.0004	0.0010	0.0012
13	1.0	30	16500	0.0395	0.0947	0.1158
16	10.0	30	16500	3.9927	9.5094	11.5111
19	0.1	100	9000	0.0001	0.0002	0.0003
22	1.0	100	9000	0.0104	0.0237	0.0279
25	10.0	100	9000	1.0255	2.3793	2.7739

```
ggplot(results, aes(factor(n), relMSE, fill = estimator)) +
  geom_col(position = "dodge") +
  facet_wrap(~ lambda, labeller = label_both) +
  scale_y_log10() +
  labs(x = "n", y = "MSE / lambda^2 (log scale)")
```



Results. ML wins in all nine cells, without exception. IQR is consistently worse than ML, and the bootstrap correction (BC) is the worst of the three in every case, and with $n = 5$ it is catastrophic: MSE ranging from

263 to over 121,000, against 0.006–62 for ML. This confirms two things from the theory: ML is the most efficient estimator when the model is correct (it uses the whole sample), and IQR loses efficiency because it only uses two quantiles. The bias correction, on the other hand, does not help here: with small n , the sample IQR can get close to zero in some bootstrap resamples, and since $\tilde{\lambda} = 2\hat{\lambda}_{IQR} - \bar{\lambda}^*$ doubles the original term, any instability in $\hat{\lambda}_{IQR}(X)$ gets amplified. With $n = 30$ and $n = 100$ the problem is much milder, but BC still does not improve on the uncorrected IQR: it trades bias for variance, and here it loses that trade.

The ranking does not change with λ , which is exactly what scale equivariance predicts: dividing MSE by λ^2 (the relMSE column), the three λ values give similar results within each n (there is some Monte Carlo noise, more visible at $n = 5$, where R hit the cap).

For reference, $\mathbb{E}[\hat{\lambda}_{ML}] = \frac{n}{n-1}\lambda$ and $\text{MSE}(\hat{\lambda}_{ML}) = \lambda^2 \frac{n+2}{(n-1)(n-2)}$ for $n > 2$, so the simulated MSE of the ML estimator should be close to that.

Exercise 2

```
frequency(ausbeer)
```

```
## [1] 4
```

```
start(ausbeer); end(ausbeer)
```

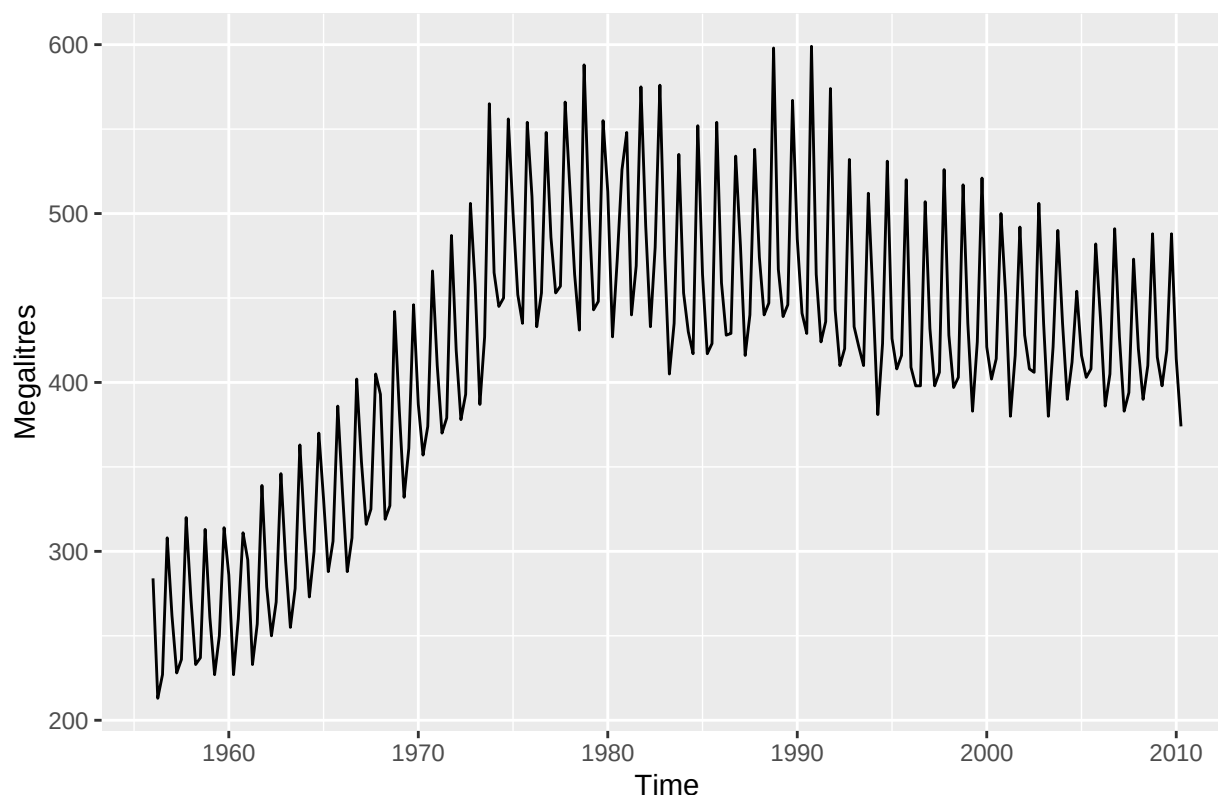
```
## [1] 1956    1
```

```
## [1] 2010    2
```

1. Frequency. It is 4, one observation per quarter (`frequency(ausbeer)` returns 4). This will also be visible in the time plot below, as a repeating up and down movement that completes once a year, so four points per cycle.

```
autoplot(ausbeer) +  
  labs(y = "Megalitres", title = "Quarterly beer production, Australia")
```

Quarterly beer production, Australia



2. Overall behaviour and trend. There is a clear long-term trend, but it is not linear. Production rises fast until the mid-1970s, then it stays roughly flat until the early 1990s, and after that it drifts down towards 2010.

4. Variability. It is not constant. The seasonal swings get bigger when the level of the series is higher, which is typical of a multiplicative pattern. A log or Box-Cox transformation should fix that.

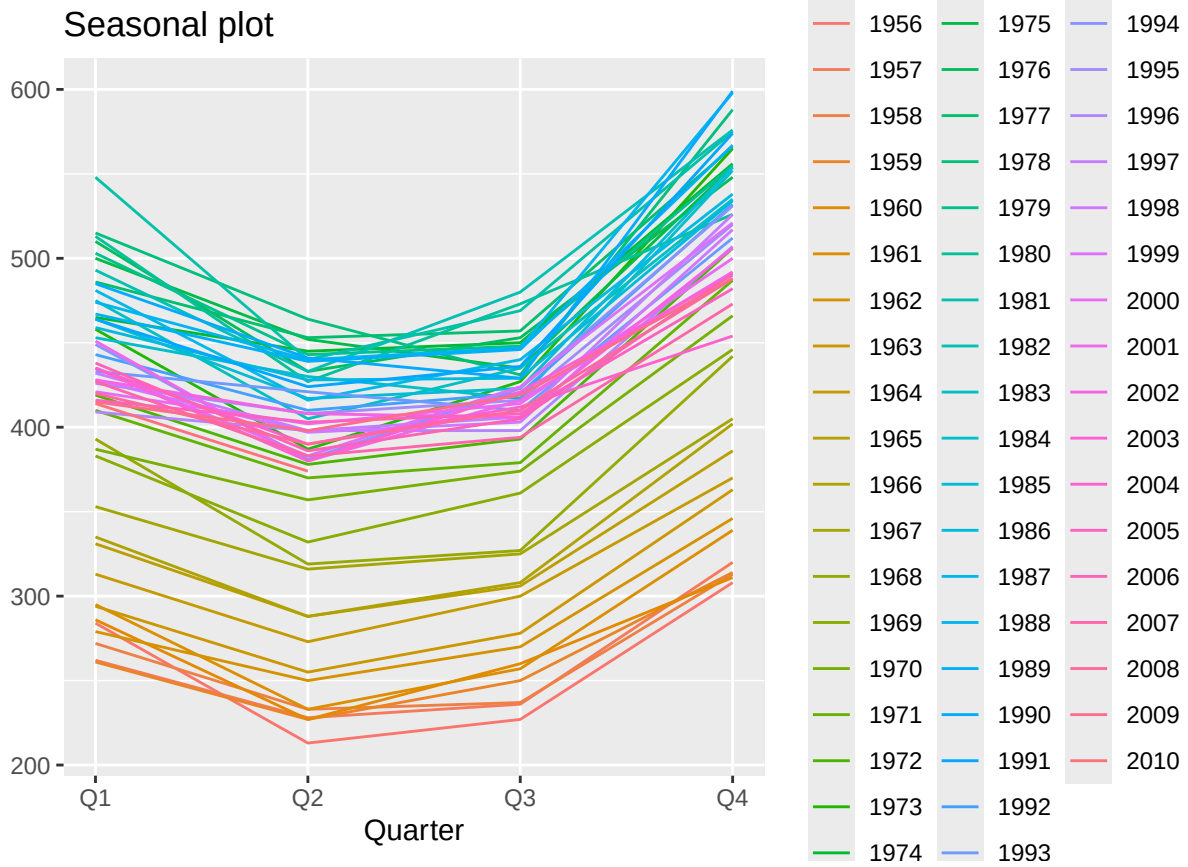
```
d <- data.frame(t = as.numeric(time(ausbeer)), y = as.numeric(ausbeer),
               q = cycle(ausbeer))
d$ratio <- d$y / stats::filter(d$y, rep(1/4, 4), sides = 2)
d[order(abs(log(d$ratio)), decreasing = TRUE)[1:6], ]
```

```
##           t    y q    ratio
## 140 1990.75 599 4 1.2505219
## 144 1991.75 574 4 1.2324208
## 132 1988.75 598 4 1.2260379
## 2   1956.25 213 2 0.8255814
## 8   1957.75 320 4 1.2064090
## 12  1958.75 313 4 1.2061657
```

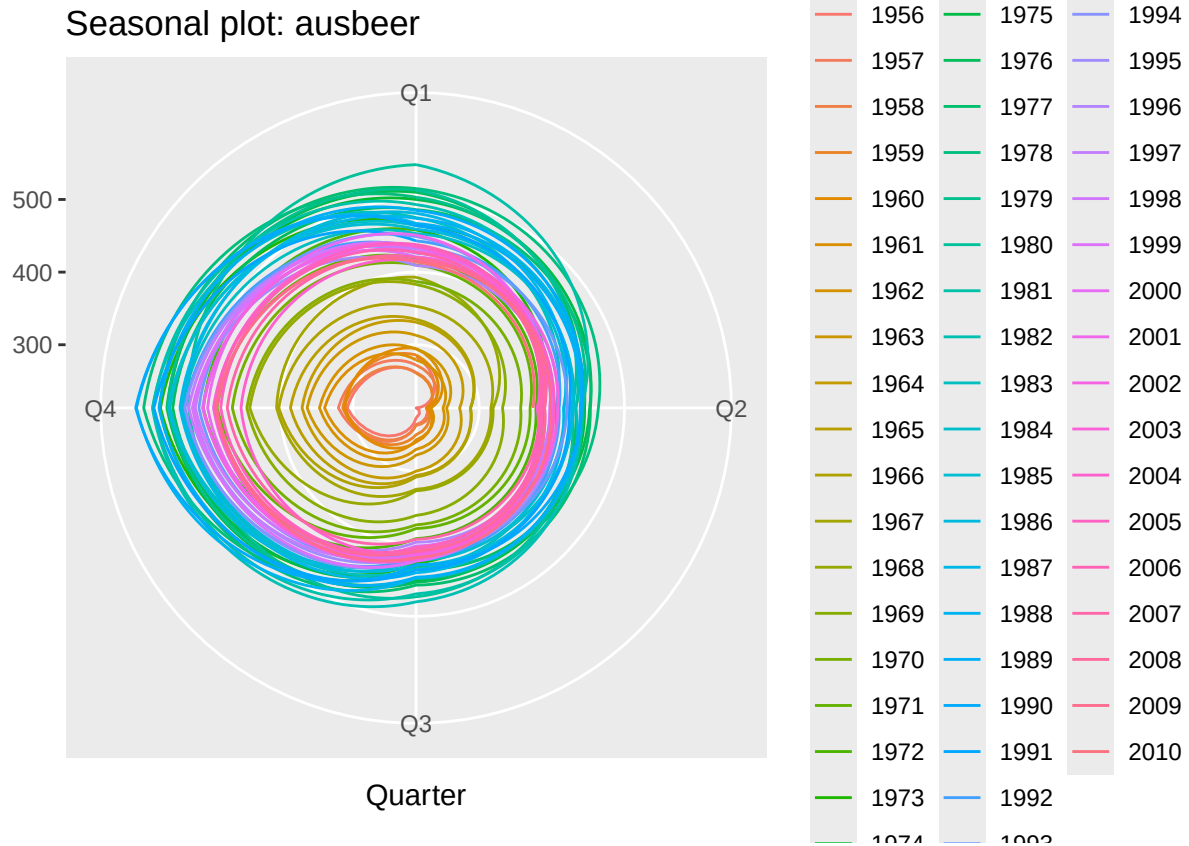
5. Unusual observations. The six that depart most from the local average are:

1988.75, 1990.75 and 1991.75 (all Q4): between 22% and 25% above the average of their neighbourhood. These coincide with the peak visible in the time plot just before production starts to decline, in the late 1980s. 1957.75 and 1958.75 (Q4): 20–21% above, at the start of the series. 1956.25 (Q2): 17% below, the only one of the six that is low rather than high.

```
ggseasonplot(ausbeer, year.labels = FALSE) + labs(title = "Seasonal plot")
```



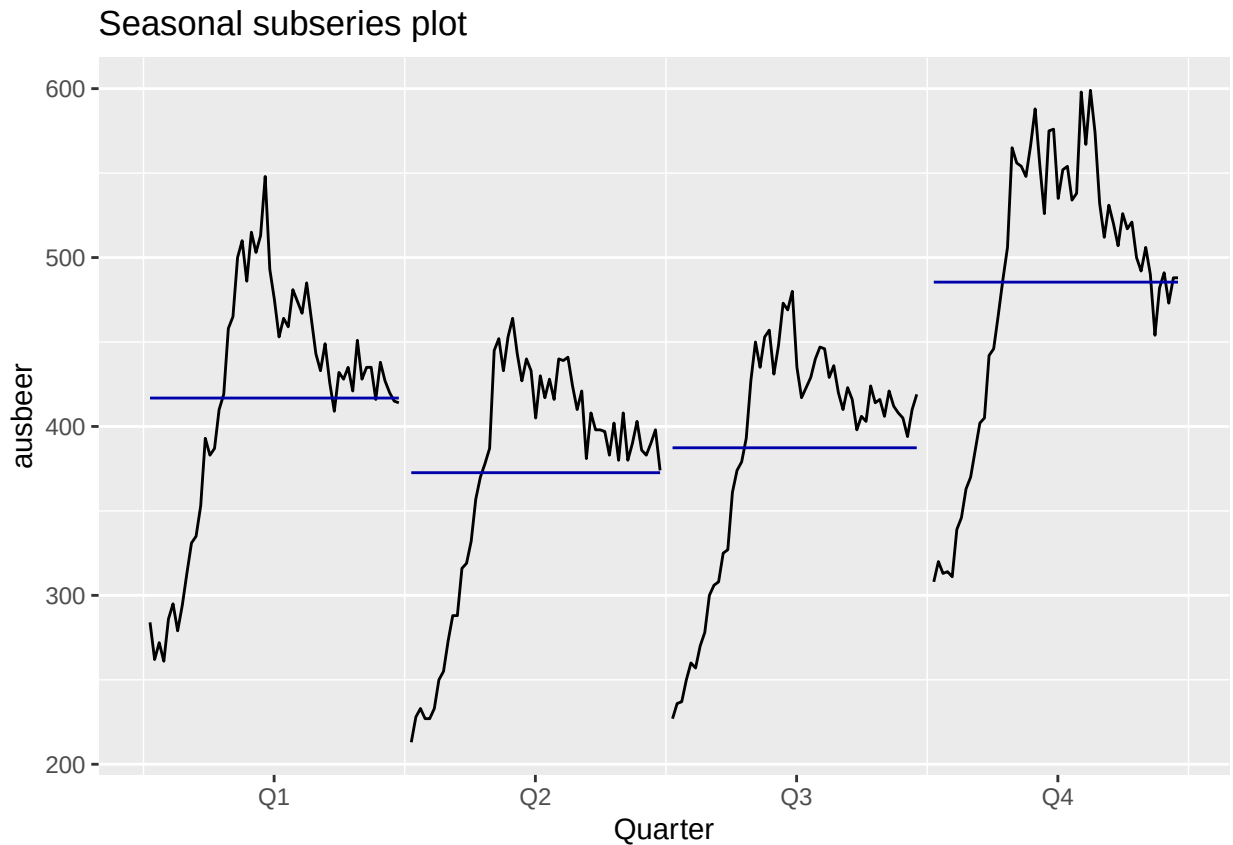
```
ggseasonplot(ausbeer, polar = TRUE)
```



3. Seasonality. Yes. The same yearly shape appears again and again: one quarter is always the peak (Q4, which is spring/summer in Australia) and Q2 (winter) is the lowest.

6. Seasonal plot. Every year is one line across the four quarters. It makes clear that the shape (dip in Q2, peak in Q4) is the same every year, and that what changes is the height of the whole line: it goes up over the years and later comes back down. In the ordinary time plot this is harder to see because the trend takes most of the space.

```
ggsubseriesplot(ausbeer) + labs(title = "Seasonal subseries plot")
```

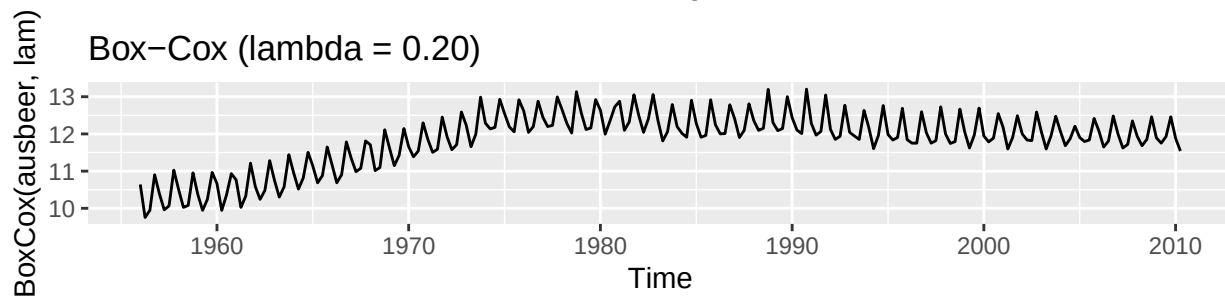
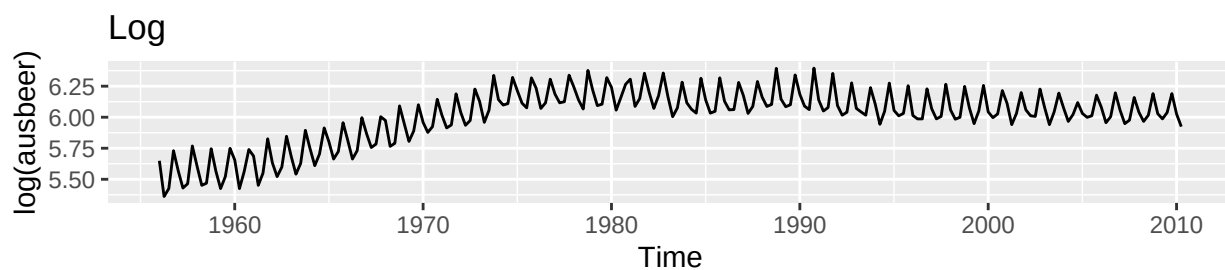
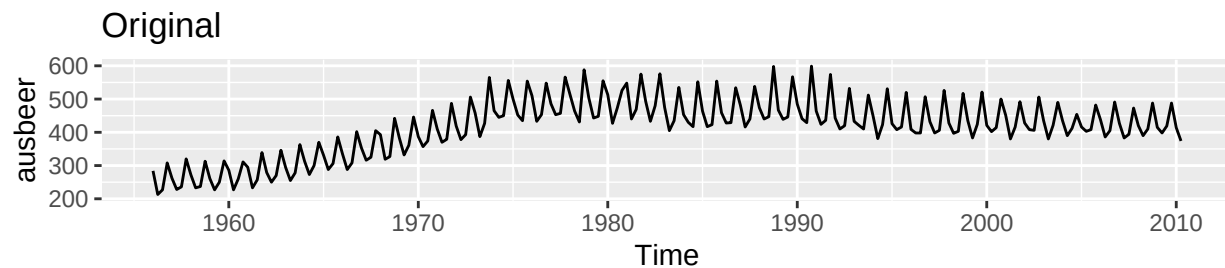


7. Subseries. There is one panel per quarter, with a horizontal line at its mean. Q4 has the highest mean and Q2 the lowest. Inside each panel the series is not flat, since all quarters go up and then down together. So the seasonal pattern is stable in shape, but the size of the differences between quarters grows with the level.

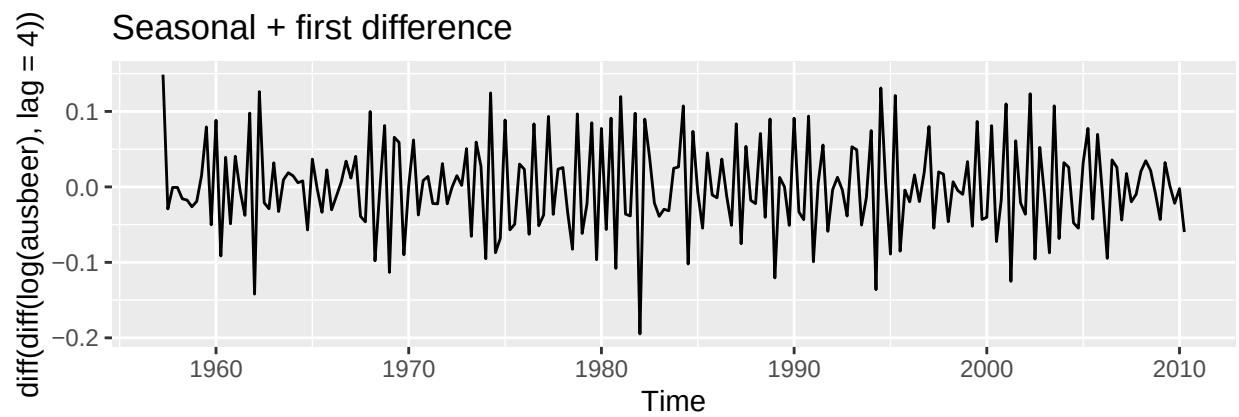
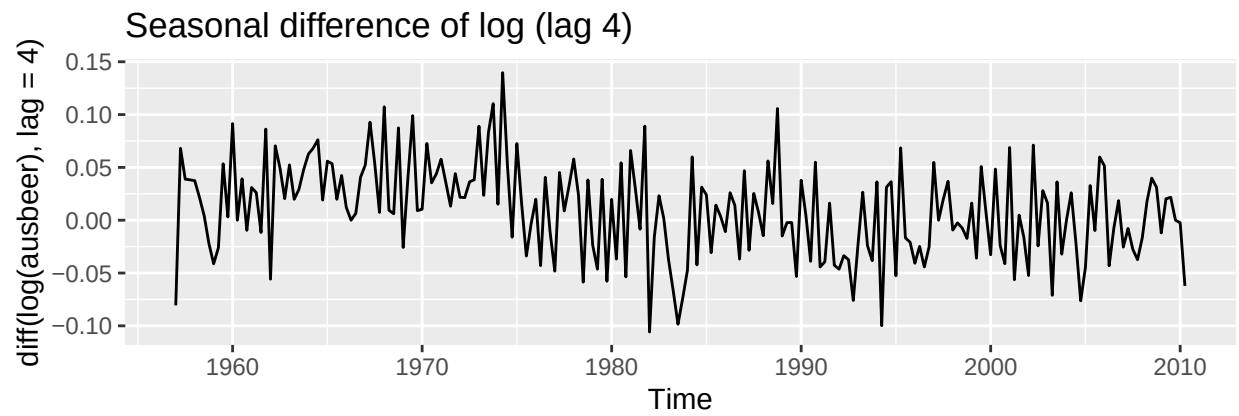
```
lam <- BoxCox.lambda(ausbeer); lam
```

```
## [1] 0.2046987
```

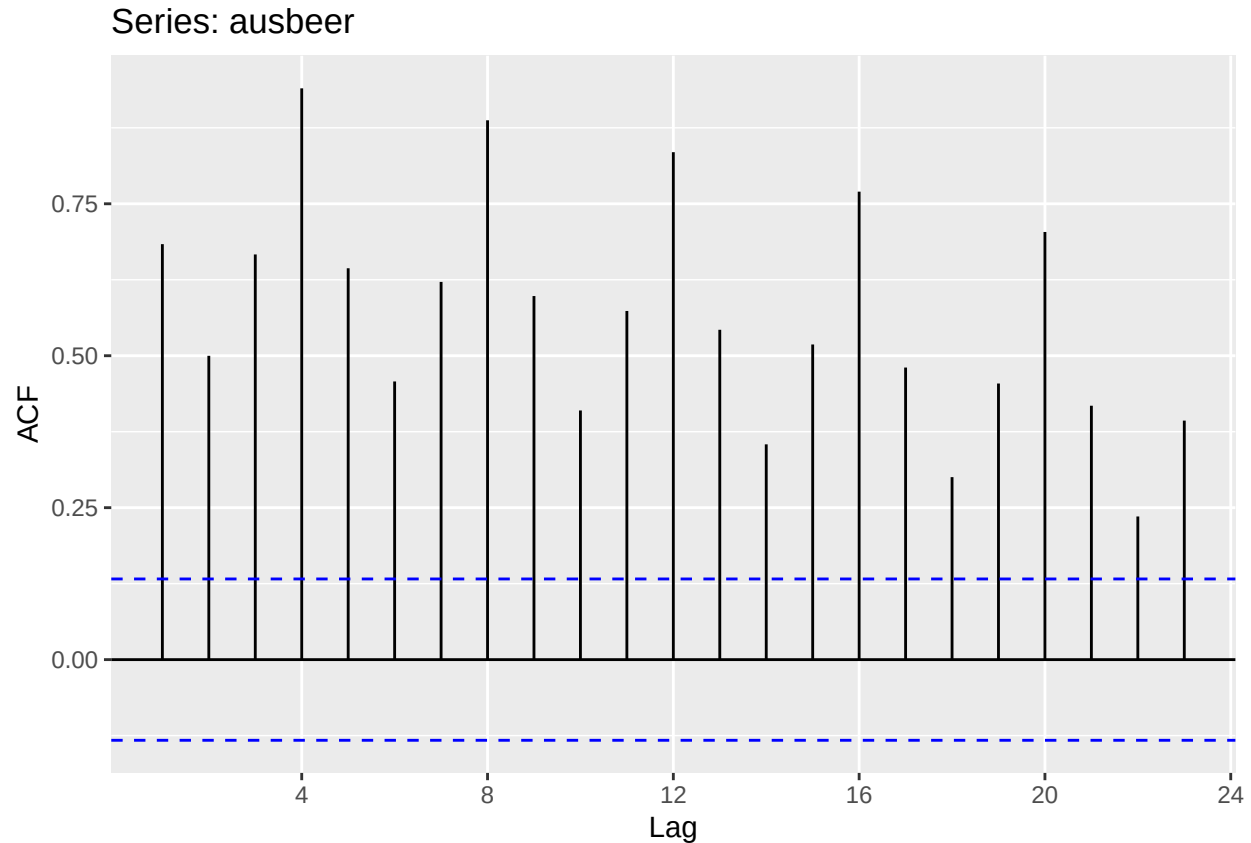
```
p1 <- autoplot(ausbeer) + ggtitle("Original")
p2 <- autoplot(log(ausbeer)) + ggtitle("Log")
p3 <- autoplot(BoxCox(ausbeer, lam)) +
  ggtitle(sprintf("Box-Cox (lambda = %.2f)", lam))
grid.arrange(p1, p2, p3, ncol = 1)
```



```
grid.arrange(
  autoplot(diff(log(ausbeer), lag = 4)) + ggtitle("Seasonal difference of log (lag 4)"),
  autoplot(diff(diff(log(ausbeer), lag = 4))) + ggtitle("Seasonal + first difference"),
  ncol = 1)
```



```
ggAcf(ausbeer)
```



8. Detrend or transform? I would do both before modelling. The trend is strong and not linear, so the series is not stationary in the mean. The seasonal amplitude also grows with the level, and the log or Box–Cox plot above shows it becomes much more even after transforming. The differenced series then look much more stable, and the ACF of the untransformed series shows large, slowly-decaying spikes at the seasonal lags (4, 8, 12...), which is more evidence that both the trend and the seasonality need to be removed before fitting a model.

References

- Hyndman, R. J., & Athanasopoulos, G. (2018). *Forecasting: Principles and practice*. OTexts. [OTexts.com/fpp2](https://otexts.com/fpp2)
- Hyndman, R. J., & Fan, Y. (1996). Sample quantiles in statistical packages. *The American Statistician*, 50(4), 361. <https://doi.org/10.2307/2684934>
- Robert, C. P., & Casella, G. (2009). *Introducing Monte Carlo methods with R*. Springer.
- Wicklin, R. (2017). Sample quantiles: A comparison of 9 definitions. SAS Blogs. <https://blogs.sas.com/content/iml/2017/05/24/definitions-sample-quantiles.html>